

SA.: Phase 4 Code Challenge: Late Show

- Due Aug 18, 2025 by 11:59pm
- Points 15
- Submitting a website url
- Available Aug 13, 2025 at 12am - Aug 29, 2025 at 11:59pm

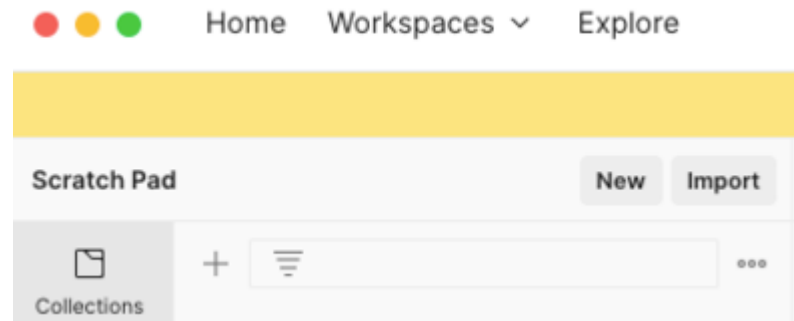
This assignment was locked Aug 29, 2025 at 11:59pm.

Instructions

For this assessment, you'll be working on an API for tracking heroes and their superpowers.

Setup

1. Create a new **PRIVATE** repository in this GitHub Organization ([Moringa-SDF-PT10](https://github.com/Moringa-SDF-PT10) (<https://github.com/Moringa-SDF-PT10>)). Ensure your repository has a name in the following format; *lateshow-firstname-lastname* (Example: **lateshow-jane-doe**). *Take note, any other repository name will **NOT** be graded.*
2. You have been provided a [Postman collection](https://moringa.instructure.com/courses/1054/files/558686?wrap=1) (<https://moringa.instructure.com/courses/1054/files/558686?wrap=1>) [↓](#) (https://moringa.instructure.com/courses/1054/files/558686/download?download_frd=1). This collection contains all the endpoints that you are required to create with this API. You can download and import it into your Postman application to test that your app works correctly.



Select 'Upload Files', navigate to this repo folder, and select 'challenge-4-lateshow.postman_collection.json' as the file to import.

Before you submit! Save and run your code to verify that it works as you expect.

You **MUST** have a well-written **README** in your repository. Ensure your markdown renders correctly before submission. You can use [Visual Studio Code Markdown preview \(https://code.visualstudio.com/docs/languages/markdown#_markdown-preview\)](https://code.visualstudio.com/docs/languages/markdown#_markdown-preview) to see how it would appear on your **GitHub** repository.

Resources

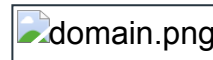
- [How to write a good README. \(https://www.freecodecamp.org/news/how-to-write-a-good-readme-file/\)](https://www.freecodecamp.org/news/how-to-write-a-good-readme-file/)

Deliverables

Your job is to build out the Flask API to add the functionality described in the deliverables below.

Models

You will implement an API for the following data model:



Now you can implement the relationships as shown in the ER Diagram:

- An `Episode` has many `Guest`'s through `Appearance`
- A `Guest` has many `Episode`'s through `Appearance`
- An `Appearance` belongs to a `Guest` and belongs to a `Episode`

Since a `Appearance` belongs to a `Episode` and a `Guest`, configure the model to cascade deletes.

Set serialization rules to limit the recursion depth.

Run the migrations and seed the database

Note that this challenge does not have a provided seed file, feel free to create your own. You can use this [CSV file](https://moringa.instructure.com/courses/1054/files/558687?wrap=1) (<https://moringa.instructure.com/courses/1054/files/558687?wrap=1>)_ ↓ (https://moringa.instructure.com/courses/1054/files/558687/download?download_frd=1) to populate the database. If you aren't able to get the provided seed file working, you are welcome to generate your own seed data to test the application.

Validations

Add validations to the `Appearance` model:

- must have a `rating` between 1 and 5 (inclusive - 1 and 5 are okay)

Routes

Set up the following routes. Make sure to return JSON data in the format specified along with the appropriate HTTP verb.

Recall you can specify fields to include or exclude when serializing a model instance to a dictionary using `to_dict()` (don't forget the comma if specifying a single field).

a. GET /episodes

Return JSON data in the format below:

```
[
  {
    "id": 1,
    "date": "1/11/99",
    "number": 1
  },
  {
    "id": 2,
    "date": "1/12/99",
```

```
    "number": 2
  }
]
```

b. GET /episodes/:id

If the `Episode` exists, return JSON data in the format below:

```
{
  {
    "id": 1,
    "date": "1/11/99",
    "number": 1,
    "appearances": [
      {
        "episode_id": 1,
        "guest": {
          "id": 1,
          "name": "Michael J. Fox",
          "occupation": "actor"
        },
        "guest_id": 1,
        "id": 1,
        "rating": 4
      }
    ]
  }
}
```

If the `Episode` does not exist, return the following JSON data, along with the appropriate HTTP status code:

```
{
  "error": "Episode not found"
}
```

c. GET /guests

Return JSON data in the format below:

```
[
  {
    "id": 1,
```

```
[
  {
    "name": "Michael J. Fox",
    "occupation": "actor"
  },
  {
    "id": 2,
    "name": "Sandra Bernhard",
    "occupation": "Comedian"
  },
  {
    "id": 3,
    "name": "Tracey Ullman",
    "occupation": "television actress"
  }
]
```

d. POST /appearances

This route should create a new `Appearance` that is associated with an existing `Episode` and `Guest`. It should accept an object with the following properties in the body of the request:

```
{
  "rating": 5,
  "episode_id": 100,
  "guest_id": 123
}
```

If the `Appearance` is created successfully, send back a response with the following data:

```
{
  "id": 162,
  "rating": 5,
  "guest_id": 3,
  "episode_id": 2,
  "episode": {
    "date": "1/12/99",
    "id": 2,
    "number": 2
  },
  "guest": {
    "id": 3,
    "name": "Tracey Ullman",
    "occupation": "television actress"
  }
}
```

If the `Appearance` is **not** created successfully, return the following JSON data, along with the appropriate HTTP status code:

```
{  
  "errors": ["validation errors"]  
}
```

Phase 4 Flask CC Rubric (1)

Criteria	Ratings					Pts
Models, Relationships, and Validations	5 pts 5 All specified models, relationships, and validations are complete and correct. Validations are used when appropriate. Variable and method names are intention-revealing.	4 pts 4 All models, relationships, and validations are complete and correct, save minor mistakes in advanced deliverables. Advanced validations are complete and add informative error messages.	3 pts 3 Models, relationships, and validations mostly complete and correct per the deliverables. Advanced validations and methods may be incomplete. May have unused or unnecessary code, possibly including duplication. May have written validations instead of using appropriate built-in validations. May implement advanced query methods with iterators instead of using built-in methods.	2 pts 2 Models, relationships, and validations attempted but incomplete or have errors. Relationships may construct the wrong relationships. Validations may be missing or applied to the wrong models. Advanced query methods incomplete or have errors.	1 pts 1 Models, relationships, and validation not started or have errors that prevent the application from running. Missing validations, or validation syntax is incorrect. May have introduced models outside the specified domain.	5 pts
Routes and REST	5 pts 5 All routes deliverables work as described in the instructions. Routing follows REST naming conventions.	4 pts 4 Nearly all routing deliverables completed, possibly with minor errors in advanced deliverables. Routing follows REST conventions.	3 pts 3 Most routing logic working as specified. Some advanced deliverables may be incomplete. Routing follows REST convention.	2 pts 2 Some routing logic implemented, but incompletely or incorrectly. May have routes actions not included in the deliverables.	1 pts 1 Routes missing, naming does not follow REST.	5 pts

Criteria	Ratings					Pts
Response Structure	5 pts 5 Includes correct JSON response body with nested data where indicated. All routes return the appropriate HTTP status code for successful requests as well as unsuccessful requests. Uses a serializer to structure data.	4 pts 4 Includes correct JSON response body with nested data where indicated. All routes return the appropriate HTTP status code for successful requests as well as unsuccessful requests.	3 pts 3 Includes appropriate top-level JSON data in response as specified. May have missing nested data structures. Includes the correct HTTP status code for successful requests. May be missing response body or status code for unsuccessful requests (not found or invalid data).	2 pts 2 Includes correct responses for some (but not all) of the routes required in the deliverables. May have missing nested data structures.	1 pts 1 Missing response or returns JSON data other than what is specified in the deliverables.	5 pts
Total Points: 15						